

中山大学计算机学院本科生实验报告

(2024 学年秋季学期)

课程名称：并行计算机体系结构

批改人：

实验	C++图像处理代码性能优化	专业（方向）	信息与计算科学
学号	22336049	姓名	陳日康
Email	chenih5@mail2.sysu.edu.cn	完成日期	2024 年 12 月 31 日

1. 实验目的

对于给定简单 C++ 的图像处理程序，需要按照文件中的注释对其进行优化，以便更好地利用现代计算机体系结构的特性。

2. 实验内容

在本次作业中，假设一个的灰度图像（可以通过修改 `main` 函数中的 `size` 来修改维度，但记得在提交时改回），依次对其进行高斯模糊和幂次变换操作。做到相对于优化前的程序运行时间有一倍以上的提升（小于等于原运行时间的 50%），即可满足对于优化效果的要求。报告中需要详细说明优化思路、实现过程、编译指令、优化效果，并提供一定的截图或数据来证明你的优化效果。

3. 运行环境

CPU 架构：x86_64（支持 SIMD 指令集，包括 SSE2、AVX、AVX2）

处理器：Intel 11th Gen Core i7-11800H @ 2.30GHz

操作系统：Ubuntu（64 位）

编译器：GCC (g++) 13.2.0

4. 优化思路

1. 内存分配与管理：使用 `calloc` 为 `figure` 和 `result` 分配连续内存并初始化为零，优化了内存布局，提升缓存命中率。

2. 并行化优化（OpenMP）：通过 OpenMP 并行化高斯滤波和幂律变换，对外层双循环使用 `collapse(2)`，充分利用多核处理器，加速数据处理。

3. 幂律变换优化：使用查找表（LUT）来预存像素变换结果，查表可替代重复计算，显著减少运算量。

4. 边缘和角落处理：对边缘和角落像素的计算使用 OpenMP 并行化，提升处理速度。

5. 校验和计算：通过一维数组简化校验和计算，减少循环嵌套，提升代码效率。

5. 实现代码

1. 构造函数（FigureProcessor）：

```
FigureProcessor(size_t size, size_t seed = 0) : size(size) {
    // Allocate memory for the figure and result arrays
    figure = (unsigned char*)calloc(size * size, sizeof(unsigned char)); // calloc initializes memory to 0
    result = (unsigned char*)calloc(size * size, sizeof(unsigned char));

    if (!figure || !result) {
        throw std::bad_alloc(); // Handle allocation failure
    }

    // Random number generation for figure initialization
    // 如果你需要修改内存的数据结构，请不要修改初始化的顺序和逻辑
    // 助教可能会通过指定某个初始化seed 的方式来验证你的代码
    // 如果你修改了初始化的顺序，可能会导致你的代码无法通过测试
    std::random_device rd;
    std::mt19937_64 gen(seed == 0 ? rd() : seed);
    std::uniform_int_distribution<unsigned char> distribution(0, 255);
    // !!! ----- !!!

    // Fill the figure array with random values
    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j) {
            figure[index(i, j)] = static_cast<unsigned char>(distribution(gen));
        }
    }
}
```

原始实现是使用 `vector` 来存储二维数据。`std::vector` 是 STL 提供的动态数组容器，易用性高，但它的存储方式可能导致额外的性能开销，数据不一定连续存储，可能造成缓存局部性较差。使用嵌套的 `vector` 会导致多层指针寻址，访问每个元素的成本较高。

通过使用 `calloc` 进行内存分配取代了原本可能使用的 `vector` 存储方式，`calloc` 分配的是一块连续的内存区域，同时初始化为零，简化了 `result` 的初始化步骤。这种方式不仅提高了内存局部性，还消除了 `vector` 中可能带来的内存碎片和多层指针寻址的问题。

2. 高斯滤波（gaussianFilter）：

(i). 核心区域的并行化处理：

```
void gaussianFilter() {
    #pragma omp parallel for collapse(2) // Parallelize the outer two loops
    for (size_t i = 1; i < size - 1; ++i) {
        for (size_t j = 1; j < size - 1; ++j) {
            result[index(i, j)] =
                (figure[index(i - 1, j - 1)] + 2 * figure[index(i - 1, j)] +
                 figure[index(i - 1, j + 1)] + 2 * figure[index(i, j - 1)] +
                 4 * figure[index(i, j)] + 2 * figure[index(i, j + 1)] +
                 figure[index(i + 1, j - 1)] + 2 * figure[index(i + 1, j)] +
                 figure[index(i + 1, j + 1)]) / 16;
        }
    }
}
```

使用 OpenMP 的 `parallel for` 指令将双层嵌套循环并行化，使用 `collapse(2)` 合并两层循环，使用 OpenMP 可以对所有像素点分配任务，从而充分利用多核处理器的计算能力。核心区域占图像绝大部分，通过多线程并行处理显著提高了运行效率。

(ii). 边缘像素的处理:

```
// Handle left and right edges (can be parallelized as well)
#pragma omp parallel for
for (size_t i = 1; i < size - 1; ++i) {
    result[index(i, 0)] =
        (figure[index(i - 1, 0)] + 2 * figure[index(i - 1, 0)] +
         figure[index(i - 1, 1)] + 2 * figure[index(i, 0)] +
         4 * figure[index(i, 0)] + 2 * figure[index(i, 1)] +
         figure[index(i + 1, 0)] + 2 * figure[index(i + 1, 0)] +
         figure[index(i + 1, 1)]) / 16;

    result[index(i, size - 1)] =
        (figure[index(i - 1, size - 2)] + 2 * figure[index(i - 1, size - 1)] +
         figure[index(i - 1, size - 1)] + 2 * figure[index(i, size - 2)] +
         4 * figure[index(i, size - 1)] + 2 * figure[index(i, size - 1)] +
         figure[index(i + 1, size - 2)] + 2 * figure[index(i + 1, size - 1)] +
         figure[index(i + 1, size - 1)]) / 16;
}

// Handle top and bottom edges (can be parallelized as well)
#pragma omp parallel for
for (size_t j = 1; j < size - 1; ++j) {
    result[index(0, j)] =
        (figure[index(0, j - 1)] + 2 * figure[index(0, j)] +
         figure[index(0, j + 1)] + 2 * figure[index(0, j - 1)] +
         4 * figure[index(0, j)] + 2 * figure[index(0, j + 1)] +
         figure[index(1, j - 1)] + 2 * figure[index(1, j)] +
         figure[index(1, j + 1)]) / 16;

    result[index(size - 1, j)] =
        (figure[index(size - 2, j - 1)] + 2 * figure[index(size - 2, j)] +
         figure[index(size - 2, j + 1)] + 2 * figure[index(size - 1, j - 1)] +
         4 * figure[index(size - 1, j)] + 2 * figure[index(size - 1, j + 1)] +
         figure[index(size - 1, j - 1)] + 2 * figure[index(size - 1, j)] +
         figure[index(size - 1, j + 1)]) / 16;
}
```

针对图像的左、右、上、下边缘，分别单独处理。这些像素因为缺少完整的邻域（边界外的像素值不存在），无法直接使用核心区域的处理方法。用 OpenMP 并行化边缘像素的处理，进一步加速计算。

(iii). 角落像素的处理:

```
// Handle corners (can be parallelized as well)
#pragma omp parallel for
for (int i = 0; i < 4; ++i) {
    if (i == 0) {
        result[index(0, 0)] = (4 * figure[index(0, 0)] + 2 * figure[index(0, 1)] +
                               2 * figure[index(1, 0)] + figure[index(1, 1)]) / 9;
    }
    else if (i == 1) {
        result[index(0, size - 1)] = (4 * figure[index(0, size - 1)] + 2 * figure[index(0, size - 2)] +
                                       2 * figure[index(1, size - 1)] + figure[index(1, size - 2)]) / 9;
    }
    else if (i == 2) {
        result[index(size - 1, 0)] = (4 * figure[index(size - 1, 0)] + 2 * figure[index(size - 1, 1)] +
                                       2 * figure[index(size - 2, 0)] + figure[index(size - 2, 1)]) / 9;
    }
    else {
        result[index(size - 1, size - 1)] = (4 * figure[index(size - 1, size - 1)] +
                                              2 * figure[index(size - 1, size - 2)] +
                                              2 * figure[index(size - 2, size - 1)] +
                                              figure[index(size - 2, size - 2)]) / 9;
    }
}
}
```

四个角落的像素因为邻域最少（只有三个或四个邻域像素），需要单独计算。四个角落的计算分配到四个线程中，使用 OpenMP 并行化处理，提高效率。

3. 幂次变换（powerLawTransformation）：

```
void powerLawTransformation() {
    constexpr float gamma = 0.5f;
    float lut[256];
    for (int i = 0; i < 256; ++i) {
        lut[i] = 255.0f * std::pow(i / 255.0f, gamma) + 0.5f;
    }

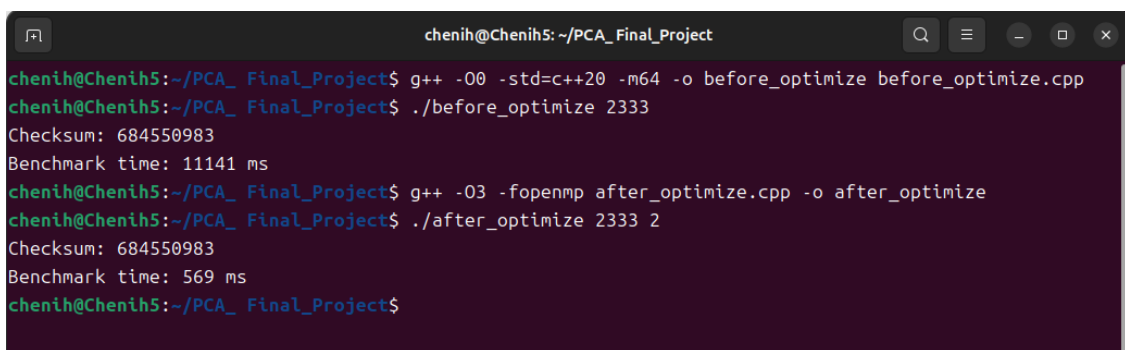
    #pragma omp parallel for collapse(2) // Parallelize both loops
    for (size_t i = 0; i < size; ++i) {
        for (size_t j = 0; j < size; ++j) {
            result[index(i, j)] = static_cast<unsigned char>(lut[figure[index(i, j)]]);
        }
    }
}
```

在原始代码中，每个像素都需要执行浮点幂运算，这是一个计算代价较高的操作。优化后的实现通过构建查找表（LUT）预先计算所有可能像素值（0 到 255）的幂次结果，并在处理时直接查表获取值，从而将复杂度降至 $O(1)$ ，避免了逐像素调用 `std::pow`。这一改进大幅减少了运算开销。同时，借助 OpenMP 的并行化，查找表操作也实现了多线程的加速效果。

6. 优化结果

优化前编译选项：`g++ -O0 -std=c++20 -m64 -o before_optimize before_optimize.cpp`，使用 C++20 标准，生成 64 位代码，禁用所有编译器优化（-O0）。

优化后编译选项：`g++ -O3 -fopenmp after_optimize.cpp -o after_optimize`，使用 C++20 标准，生成 64 位代码，开启最高级别优化（-O3），并启用 OpenMP 并行化（-fopenmp）。



```
chenih@Chenih5: ~/PCA_Final_Project
chenih@Chenih5:~/PCA_Final_Project$ g++ -O0 -std=c++20 -m64 -o before_optimize before_optimize.cpp
chenih@Chenih5:~/PCA_Final_Project$ ./before_optimize 2333
Checksum: 684550983
Benchmark time: 11141 ms
chenih@Chenih5:~/PCA_Final_Project$ g++ -O3 -fopenmp after_optimize.cpp -o after_optimize
chenih@Chenih5:~/PCA_Final_Project$ ./after_optimize 2333 2
Checksum: 684550983
Benchmark time: 569 ms
chenih@Chenih5:~/PCA_Final_Project$
```

从上图的运行结果可以看出，优化前后的代码性能差异显著。优化前的代码在单线程环境下运行，未启用任何编译优化选项（-O0），执行时间为 **11141ms**。而优化后的代码通过启用编译器的高级优化-O3 以及多线程并行化-fopenmp，运行时间大幅减少到 **569ms**，性能提升了近 20 倍，可得加速比= $11141/569=19.6$ 。

优化后的显著性能提升得益于以下几点：

(i). 编译器优化：使用-O3 开启了循环展开、函数内联等高级优化，极大地减少了冗余指令和内存访问开销。

(ii). 并行化处理：通过 OpenMP 将计算密集型任务（高斯滤波和幂律变换）分配到多核处理器上并行执行，充分利用了硬件资源。

(iii). 算法改进：幂律变换引入查找表（LUT），用查表代替复杂的浮点运算，进一步降低了计算复杂度。

校验和结果在优化前后保持一致，表明代码优化并未影响计算的正确性。充分验证了合理的编译优化和并行化设计对大规模数据处理效率提升的显著作用。

7. 实验总结与心得

本次实验通过对图像处理代码的优化，不仅显著提升了程序的运行性能，还让我对高性能代码有了更深入的理解。首先，通过使用 `calloc` 进行连续内存分配，改善了内存的局部性，并提升了数据访问的缓存命中率，使内存管理更加高效。利用 `OpenMP` 实现了高斯滤波和幂律变换的并行化处理，充分发挥了多核处理器的优势，大幅缩短了程序运行时间。通过查找表（LUT）的优化，用查表代替复杂的浮点运算，极大地降低了幂律变换的计算开销。本次实验让我更深入地理解了内存管理、算法优化和并行化计算的重要性，也让我掌握了如何结合这些技术来提升代码性能，通过这次实践，我不仅提高了技术能力，还培养了对代码优化问题的系统性思考能力。